

```

def __init__(self, usePrettyBoard=False, useLogging=False):
    """Create a new, blank tic tac toe board."""
    self._spaces = {} # The board is represented as a Python dictionary.
    for space in ALL_SPACES:
        self._spaces[space] = BLANK # All spaces start as blank.

def getBoardStr(self):
    """Return a text-representation of the board."""
    return f'''
{self._spaces['1']}|{self._spaces['2']}|{self._spaces['3']}  1 2 3
-+-+
{self._spaces['4']}|{self._spaces['5']}|{self._spaces['6']}  4 5 6
-+-+
{self._spaces['7']}|{self._spaces['8']}|{self._spaces['9']}  7 8 9'''

def isValidSpace(self, space):
    """Returns True if the space on the board is a valid space number
    and the space is blank."""
    return space in ALL_SPACES and self._spaces[space] == BLANK

def isWinner(self, player):
    """Return True if player is a winner on this TTTBoard."""
    s, p = self._spaces, player # Shorter names as "syntactic sugar".
    # Check for 3 marks across the 3 rows, 3 columns, and 2 diagonals.
    return ((s['1'] == s['2'] == s['3'] == p) or # Across the top
            (s['4'] == s['5'] == s['6'] == p) or # Across the middle
            (s['7'] == s['8'] == s['9'] == p) or # Across the bottom
            (s['1'] == s['4'] == s['7'] == p) or # Down the left
            (s['2'] == s['5'] == s['8'] == p) or # Down the middle
            (s['3'] == s['6'] == s['9'] == p) or # Down the right
            (s['3'] == s['5'] == s['7'] == p) or # Diagonal
            (s['1'] == s['5'] == s['9'] == p)) # Diagonal

def isBoardFull(self):
    """Return True if every space on the board has been taken."""
    for space in ALL_SPACES:
        if self._spaces[space] == BLANK:
            return False # If a single space is blank, return False.
    return True # No spaces are blank, so return True.

def updateBoard(self, space, player):
    """Sets the space on the board to player."""
    self._spaces[space] = player

```